🚀 CollabPro - Executive Summary

📌 NE OLACAK?

CollabPro, gerçek zamanlı takım işbirliği platformu. Notion + Figma + Asana'nın kombinasyonu gibi düşün.

Basit tanım:

> Uzaktan çalışan takımların doküman, proje ve task'ları real-time olarak beraber yönetebilecekleri web uygulaması.

🎯 CORE FEATURES

✨ 1. Real-Time Collaboration

Birden fazla kullanıcı aynı dokümanda aynı anda çalışabiliyor.

- Birisi yazı yazıyor → diğeri anında görüyor

- Typing indicator ("Ali yazıyor...")

- Live presence (kim online)

- Conflict resolution (iki kişi aynı yazı silerse ne olur?)

🔐 2. Multi-Tenant Architecture

Farklı şirketlerin/ekiplerin kendi workspace'leri var.

- Her organization kendi veriside ayrı

- Row-Level Security (PostgreSQL'de tanımlanmış)

- Hiçbir veri leak yok

👥 3. Permission System (RBAC)

Admin: Herşeyi yapabilir + team manage + audit logs

Editor: Döküman edit + comment yazabilir

Viewer: Sadece okuyabilir

💬 4. Comments & Threads

- Dokümandaki herhangi bir yere comment yapabilir

- Reply/thread sistemi

- @mention'lar notify'lıyor

- Email notification'lar

📋 5. Audit Trail

- Her işlem log'lanıyor: kim ne yaptı, ne zaman

- Dokümanda kim silebilir? Görebilirsin.

- Compliance (GDPR, ISO gibi)

🔌 6. API for Integrations

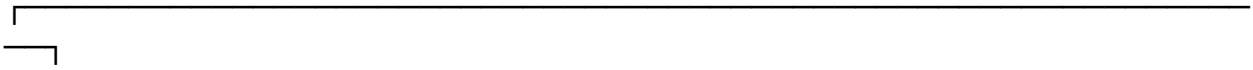
- Slack integration

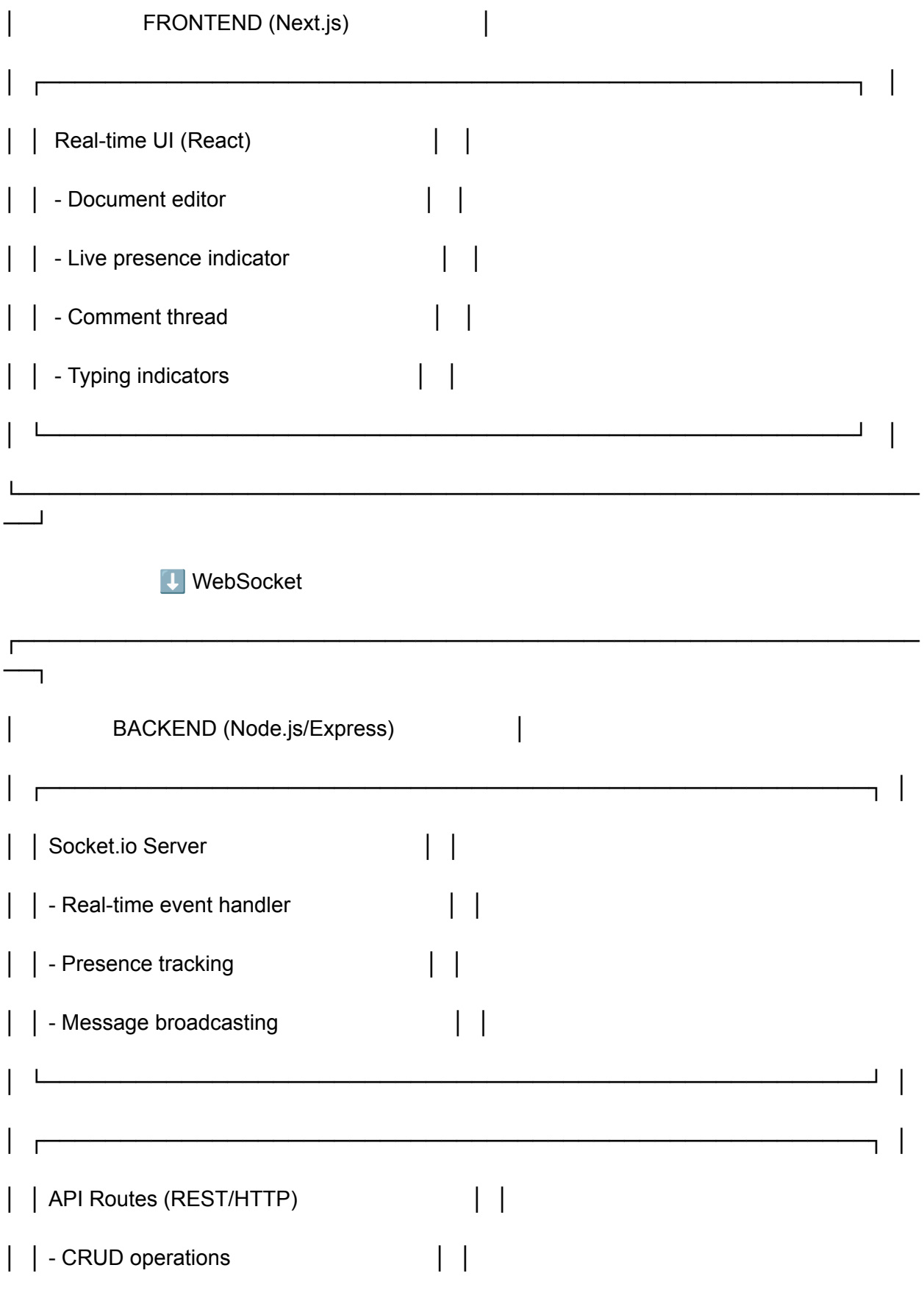
- Zapier/Make.com support

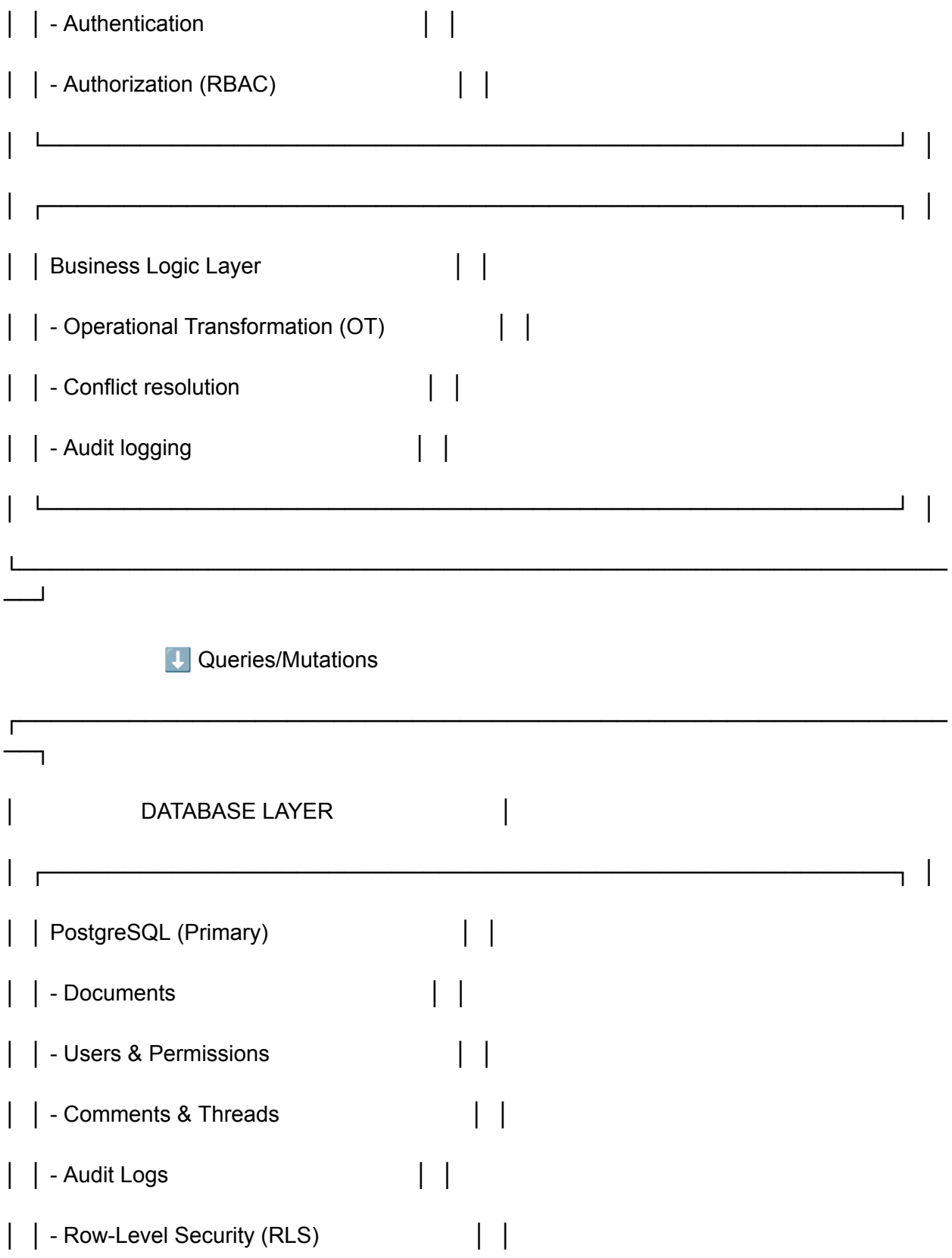
- Third-party apps bağlayabilir

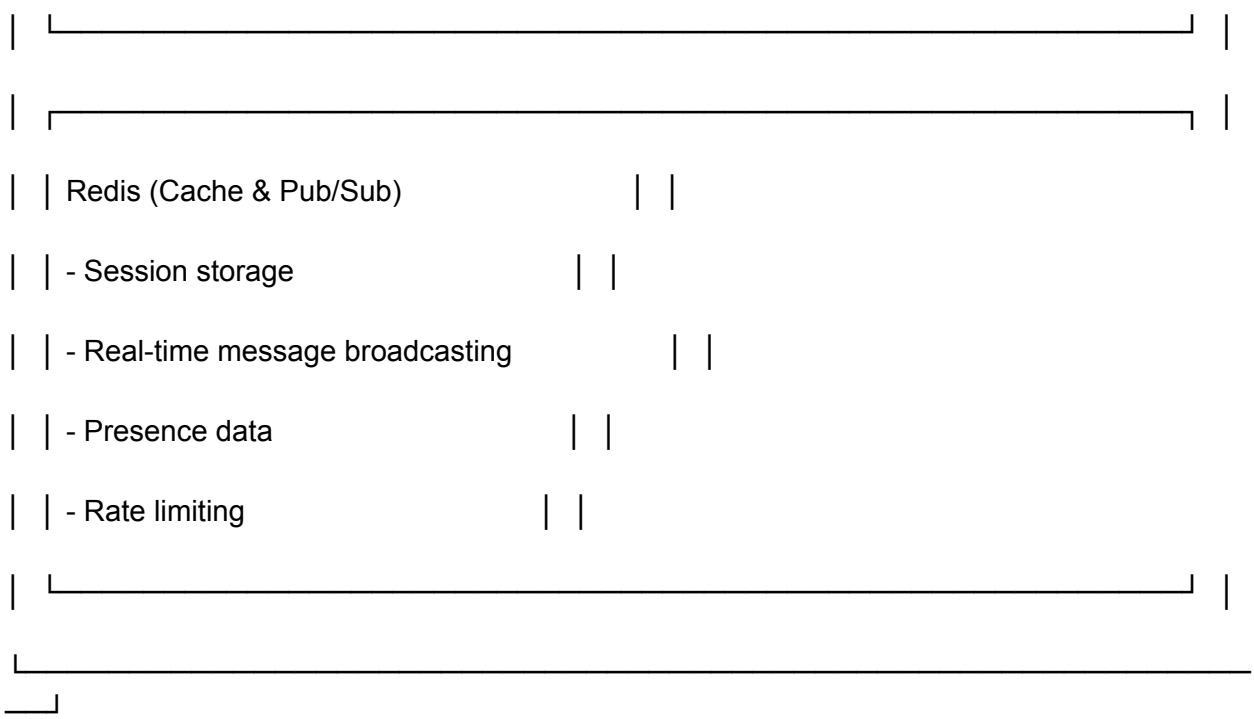
- Webhooks (olaylar harici servislere bildir)

🏗️ ARCHITECTURE OVERVIEW




FRONTEND (Next.js)
Real-time UI (React)
- Document editor
- Live presence indicator
- Comment thread
- Typing indicators
⬇️ WebSocket
BACKEND (Node.js/Express)
Socket.io Server
- Real-time event handler
- Presence tracking
- Message broadcasting
API Routes (REST/HTTP)
- CRUD operations


- Authentication
- Authorization (RBAC)
Business Logic Layer
- Operational Transformation (OT)
- Conflict resolution
- Audit logging
⬇️ Queries/Mutations
DATABASE LAYER
PostgreSQL (Primary)
- Documents
- Users & Permissions
- Comments & Threads
- Audit Logs
- Row-Level Security (RLS)



💻 TECH STACK

Layer	Technology	Reason
Frontend	Next.js 15 + React	SSR, Server Components, performance
Styling	Tailwind CSS + shadcn/ui	Production-ready components
Real-time	Socket.io + Redis adapter	WebSocket scaling
Database	PostgreSQL + Prisma	Type-safe ORM, RLS support
Auth	NextAuth.js + JWT	Secure, sessions, social login
Cache	Redis (Upstash)	Session, pub/sub, rate limiting
Monitoring	Sentry + DataDog	Error tracking, APM
Testing	Jest + Vitest + React Testing Library	Unit, integration tests

Layer	Technology	Reason
Deployment	Vercel + Render/Railway	Serverless frontend + backend

---

📊 USER JOURNEY

Scenario 1: New User

1. Ziyaretçi collab.pro'ya girer

2. "Sign Up" butonuna tıklar (email/GitHub)

3. Email verify eder

4. Organization adı girer (örn: "Acme Corp")

5. First workspace oluşturulur

6. Empty state'i görür ("Create your first document")

Scenario 2: Collaboration

1. Ali bir dokümant oluşturur ("Q4 Planning")

2. Zeynep'i invite eder (permission: Editor)

3. Zeynep email linkini tıklar → dokümant açılır

4. İkisi aynı anda edit yapıyor

5. Ali yazı yazıyor → Zeynep anında görüyor

6. Zeynep typing indicator'ı Ali'de çıkıyor

7. Zeynep başka bölüme comment yapıyor

8. Ali notification alıyor ("Zeynep commented on...")

Scenario 3: Permission Check

1. Müşteri (Viewer role) dokümantı açıyor

2. Edit button'ları disabled (görmüyor)

3. Sadece okuyabilir ve comment yapabilir

4. Database'de RLS policy kontrol ediyor:

   "SELECT * FROM documents WHERE org_id = ? AND permission >= viewer_role"

Scenario 4: Audit Log

1. Admin "Audit" sekmesine girer

2. Tüm user actions'ları görür:

   - 09:15 | Ali | Created document "Q4 Planning"

   - 09:22 | Zeynep | Edited document "Q4 Planning"

   - 09:30 | Zeynep | Added comment (line 45)

   - 09:35 | Ali | Deleted comment

4. İster CSV export edebilir

🎯 BUSINESS MODEL

Pricing Tiers

Tier	**Price**	**Features**
Free	$0	1 organization, 5 docs, 2 users
Pro	$29/mo	10 orgs, unlimited docs, 10 users, API access

Tier	Price	Features
Enterprise	Custom	SSO, audit logs, advanced permissions, SLA

Revenue

Target: 1000 Pro subscribers @ $29/mo = $29K/mo = $348K/year

Freemium conversion rate: 5% (industry standard)

📈 KEY METRICS (Success Criteria)

DAU (Daily Active Users): Target 5K

MAU (Monthly Active Users): Target 20K

Churn Rate: <5% (monthly)

Real-time update latency: <100ms (p95)

Uptime: 99.9%

API availability: 99.95%

Cost per user: <$0.50/month

🗂️ DATABASE SCHEMA

```
-- Organizations (Multi-tenancy)

CREATE TABLE organizations (

  id UUID PRIMARY KEY,

  name VARCHAR(255) NOT NULL,

  owner_id UUID NOT NULL,
```

```
  subscription_tier VARCHAR(50), -- free, pro, enterprise
  created_at TIMESTAMP DEFAULT NOW()
);
-- Users
CREATE TABLE users (
  id UUID PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255),
  password_hash VARCHAR(255),
  oauth_provider VARCHAR(50), -- github, google
  created_at TIMESTAMP DEFAULT NOW()
);
-- Team Members (org_id + user_id + role)
CREATE TABLE team_members (
  id UUID PRIMARY KEY,
  org_id UUID NOT NULL REFERENCES organizations(id),
  user_id UUID NOT NULL REFERENCES users(id),
  role VARCHAR(50) NOT NULL, -- admin, editor, viewer
  created_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(org_id, user_id)
);
-- Documents
```

```
CREATE TABLE documents (
  id UUID PRIMARY KEY,
  org_id UUID NOT NULL REFERENCES organizations(id),
  title VARCHAR(255) NOT NULL,
  content TEXT, -- or JSONB for rich text
  created_by UUID NOT NULL REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
-- Comments & Threads
CREATE TABLE comments (
  id UUID PRIMARY KEY,
  doc_id UUID NOT NULL REFERENCES documents(id),
  user_id UUID NOT NULL REFERENCES users(id),
  content TEXT NOT NULL,
  line_number INT, -- which line in document
  parent_comment_id UUID REFERENCES comments(id), -- for threads
  created_at TIMESTAMP DEFAULT NOW()
);
-- Audit Logs (Compliance)
CREATE TABLE audit_logs (
```

```
  id UUID PRIMARY KEY,
  org_id UUID NOT NULL REFERENCES organizations(id),
  user_id UUID NOT NULL REFERENCES users(id),
  action VARCHAR(100), -- create, update, delete, comment
  resource_type VARCHAR(50), -- document, comment, user
  resource_id UUID,
  changes JSONB, -- what changed
  created_at TIMESTAMP DEFAULT NOW()
);
-- Row-Level Security Policies
CREATE POLICY "users_can_see_own_org_docs"
  ON documents
  FOR SELECT
  USING (
    org_id IN (
      SELECT org_id FROM team_members WHERE user_id = current_user_id
    )
  );
```

🔐 SECURITY FEATURES

Authentication

✅ NextAuth.js (email + GitHub/Google)

✅ JWT tokens (httpOnly cookies)

✅ Session refresh tokens

✅ Password hashing (bcrypt)

Authorization

✅ RBAC (Role-Based Access Control)

- Admin: Full access + team management

- Editor: Edit + comment

- Viewer: Read-only

✅ RLS (Row-Level Security)

- PostgreSQL policies enforce access at DB level

- No data can leak via direct DB query

Data Protection

✅ HTTPS only (TLS 1.3)

✅ CORS restrictive

✅ Rate limiting (100 req/min per user)

✅ Input validation + sanitization

✅ XSS prevention (React's built-in escaping)

✅ CSRF tokens

Compliance

✅ Audit logging (who did what, when)

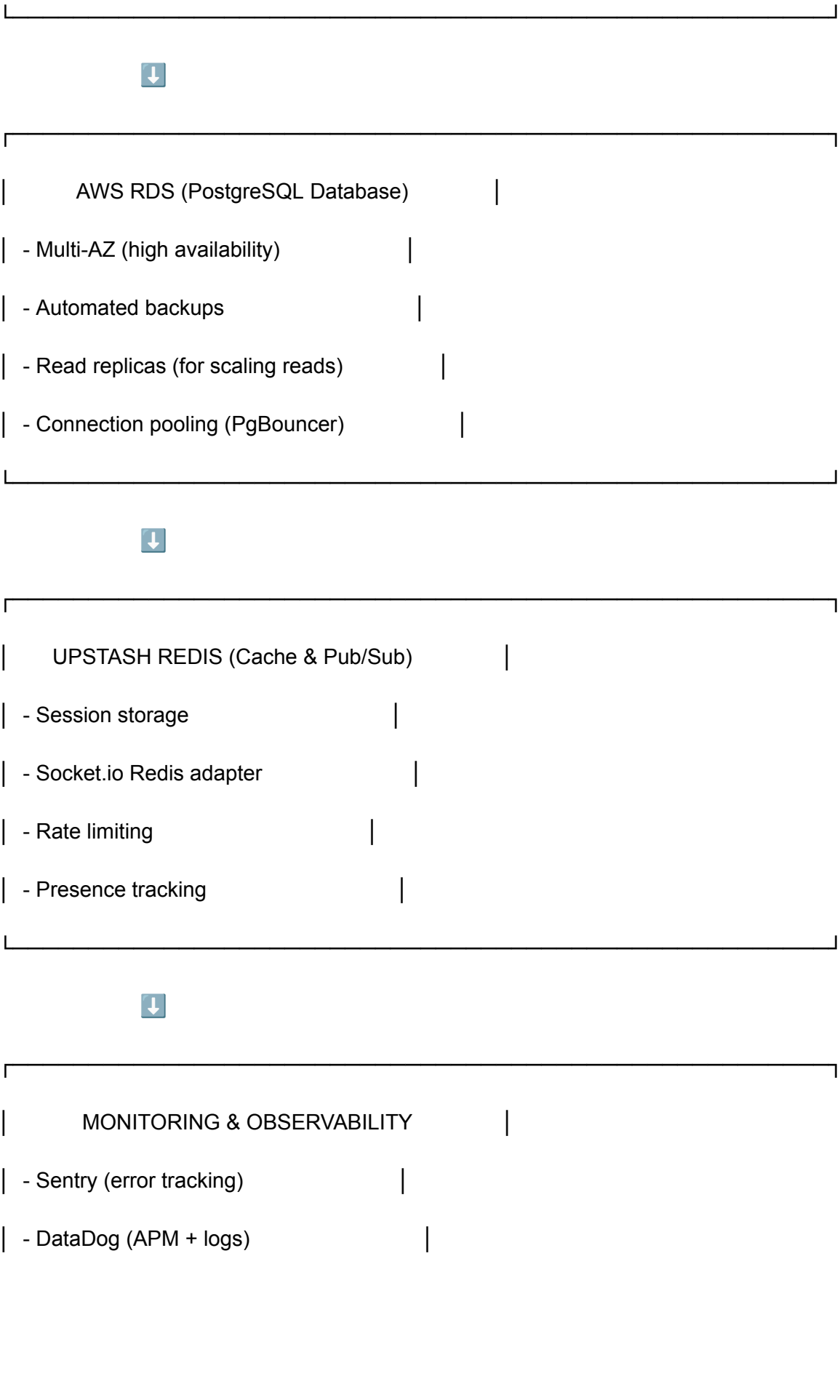
✅ Data export (GDPR right to be forgotten)

✅ Encryption at rest (AWS KMS)

✅ SOC 2 Type II ready

🚀 DEPLOYMENT ARCHITECTURE

```
┌─────────────────────────────────────────────────────────┐
│                    VERCEL (Frontend)                    │
│  - Next.js App deployed                                 │
│  - Edge middleware (auth checks)                        │
│  - Automatic preview deployments                        │
│  - CDN (image, asset delivery)                          │
└─────────────────────────────────────────────────────────┘
                          ⬇️
┌─────────────────────────────────────────────────────────┐
│                RENDER/RAILWAY (Backend)                 │
│  - Node.js/Express server                               │
│  - Socket.io service                                    │
│  - Environment: production, staging, development        │
│  - Auto-scaling (CPU/memory based)                      │
```


⬇️
AWS RDS (PostgreSQL Database)
- Multi-AZ (high availability)
- Automated backups
- Read replicas (for scaling reads)
- Connection pooling (PgBouncer)
⬇️
UPSTASH REDIS (Cache & Pub/Sub)
- Session storage
- Socket.io Redis adapter
- Rate limiting
- Presence tracking
⬇️
MONITORING & OBSERVABILITY
- Sentry (error tracking)
- DataDog (APM + logs)

```
│  - Uptime monitoring (Pingdom)                    │
└──────────────────────────────────────────────────────┘
```

📚 API ENDPOINTS (Preview)

```
# Authentication

POST   /api/v1/auth/signup          → Create account

POST   /api/v1/auth/login           → Login

POST   /api/v1/auth/logout          → Logout

POST   /api/v1/auth/refresh         → Refresh token

# Documents

GET    /api/v1/documents            → List documents

POST   /api/v1/documents            → Create document

GET    /api/v1/documents/:id        → Get document

PUT    /api/v1/documents/:id        → Update document

DELETE /api/v1/documents/:id        → Delete document

# Permissions

GET    /api/v1/documents/:id/permissions  → List permissions

POST   /api/v1/documents/:id/permissions  → Add user access

DELETE /api/v1/documents/:id/permissions/:userId → Remove access

# Comments

GET    /api/v1/documents/:id/comments     → Get comments
```

POST /api/v1/documents/:id/comments → Add comment

Audit Logs

GET /api/v1/organizations/:org_id/audit-logs → Get audit trail

Organizations

GET /api/v1/organizations → List orgs (user's)

POST /api/v1/organizations → Create org

PUT /api/v1/organizations/:id → Update org

🎓 LEARNING OUTCOMES (Senin için)

Bu projeyi yaparak öğreneceklerin:

✅ Real-Time Systems

- WebSocket vs HTTP trade-offs

- Socket.io scaling (Redis adapter)

- Presence tracking algorithms

- Typing indicators best practices

- Conflict resolution (Operational Transformation basics)

✅ Database Design

- Multi-tenancy patterns

- Row-Level Security (PostgreSQL)

- Schema design for large scale

- Indexing strategies

- Connection pooling

✅ Security & Compliance

- RBAC implementation

- Authentication flows

- Audit logging patterns

- Data protection regulations

- OWASP top 10 prevention

✅ Production Systems

- Error tracking (Sentry)

- Performance monitoring (DataDog)

- CI/CD pipelines

- Deployment strategies

- Cost optimization

✅ Testing

- Unit tests (Jest)

- Integration tests (API)

- E2E tests (Playwright)

- Socket.io testing strategies

- Mocking external services

⏱️ TIMELINE

WEEK 1 (Foundation + Real-Time)

├─ Day 1: Project scaffold, Next.js setup, DB schema

├─ Day 2: Authentication (NextAuth.js)

├─ Day 3: Socket.io + Redis setup

├─ Day 4: Real-time document sync + presence tracking

└─ Day 5: Typing indicators + basic tests

WEEK 2 (Collaboration + RBAC)

├─ Day 1: Comment system + threads

├─ Day 2: RBAC middleware + RLS policies

├─ Day 3: Audit logging system

├─ Day 4: API versioning + OpenAPI docs

└─ Day 5: Error handling + Sentry integration

WEEK 3 (Enterprise + Deployment)

├─ Day 1: DataDog monitoring setup

├─ Day 2: Comprehensive test suite

├─ Day 3: Security audit + optimization

├─ Day 4: GitHub + CI/CD pipeline

└─ Day 5: Vercel/Render deployment + demo

TOTAL: 15 days (3 weeks)

💡 WHY THIS PROJECT?

For Portfolio:

✅ Real-time systems = valuable skill

✅ Multi-tenant = enterprise pattern

✅ Full-stack = shows depth & breadth

✅ Production-ready = not toy project

✅ GitHub star magnet

For Interviews:

Q: "Real-time systems nasıl design ederdin?"

A: "CollabPro yaptım, şöyle..." (live demo + code)

Q: "Multi-tenant SaaS'ı nasıl build ederdin?"

A: "RLS + RBAC + audit logs..." (architectural deep dive)

Q: "Büyük skalada ne düşünürsün?"

A: "Redis adapter, connection pooling, caching strategy..."

For Your Career:

✅ Wanderlens → CollabPro = progression

✅ Logitrack v2 → CollabPro = lateral skill building

✅ Job applications: "Built production SaaS platform"

✅ Salary negotiation: "Worked on multi-tenant architecture"

🎯 SUCCESS CRITERIA

Project başarılı = eğer:

- ☐ Live uygulamaya Vercel'de erişilebiliyor
- ☐ Real-time collab sub-100ms latency
- ☐ 50+ test cases pass (%80+ coverage)
- ☐ Audit logs working

- [ ] API documented (OpenAPI)
- [ ] GitHub repo 100+ stars (stretch)
- [ ] Demo video + blog post yazılmış
- [ ] Interview'da anlatabiliyorsun deep details

📞 NEXT STEPS

1. **Proje onayı:** "Tamam, CollabPro ile başlayalım"
2. **Environment setup:** Node, PostgreSQL, Redis kurulumu
3. **Repository init:** GitHub repo oluştur
4. **Day 1 kickoff:** Scaffold + DB schema + auth
5. **Daily standups:** Progress tracking

Ready to build? 🚀